Tutorial/Laboratory 05

Developing Server-side Web Applications with PHP

1. LEARNING OUTCOMES

Upon completion of these laboratory exercises, you should be able to:

- Build server-side web applications with PHP.
- Debug PHP pages using the Xdebug extension.
- Convert a HTML project into a PHP project.
- Organize and reuse content with PHP includes.

2. REQUIRED SOFTWARE

- Visual Studio Code (https://code.visualstudio.com/)
- PHP 8.2 (or later): (https://www.php.net/downloads)
- PHP Debug extension for VS Code (https://marketplace.visualstudio.com/items?itemName=xdebug.php-debug)
- Xdebug 3.2.2 (or later): (https://xdebug.org/download/historical)

3. EXERCISE 1: INSTALL PHP AND CONFIGURE VS CODE

3.1 Unlike pure client-side applications written in HTML and JavaScript, PHP files cannot be read/executed directly by a web browser. The website must be run on a web server, which can be local or remote.

In a production environment you would install PHP on a remote web server, typically a "LAMP" stack, which we'll cover in a future Lab. But for development purposes, we can install PHP on our local machine and execute our applications using PHP's built-in web server running on localhost.

3.2 To install PHP on Windows:

a. Download the current version of PHP (8.2.10 or later) from the following site: https://windows.php.net/download. Since the local web server that we're using is single-threaded, you should select the "Non Thread Safe" 64-bit version (**VS16 x64 Non Thread Safe**). Click on the Zip file link to download the file:

Note: the VC16 build requires the Visual C++ Redistributable for Visual Studio 2015-2019 x64, which most Windows computers should have. If yours does not, you can download and run the installer from here.

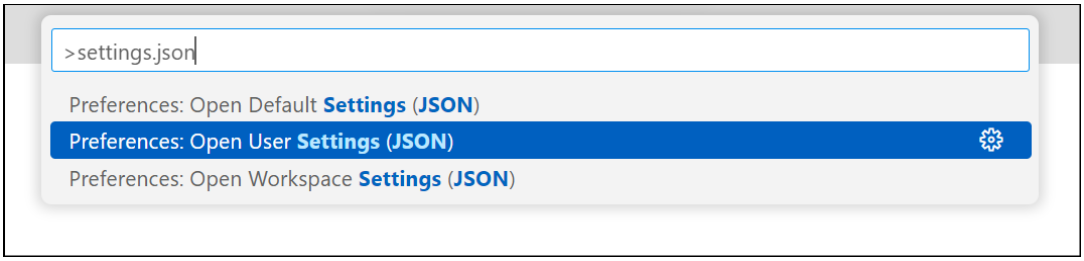
b. Unzip the file and copy the contents to an easily accessible location on your local drive, e.g., `D:\Dev\php`. You'll need to remember this location for subsequent tasks.

3.3 To install PHP on Mac OS, refer to the instructions at: https://www.php.net/manual/en/install.macosx.php. For other Linux-type operating systems, refer to https://www.php.net/downloads.

If you encounter any issues, you may refer to the PHP documentation at: https://www.php.net/manual/en/install.php.

3.4 Next, install the PHP Debug extension for Visual Studio Code (https://marketplace.visualstudio.com/items?itemName=xdebug.php-debug). This extension allows us to run our PHP applications on a local built-in web server and to remotely debug our code (we'll learn how debug in the next exercise).

3.5 Once PHP is installed, we need to let VS Code know where to find the PHP executable file (`php.exe`). Type **Ctrl + Shift + P** to access the Command Palette, then type "settings. Json" in the search bar and select User Settings:



This will open the `settings.json` file for editing. Near the bottom of this file, add configuration settings for `php.validate.executablePath` and `php.debug.executablePath`, as shown below. Be sure to use the correct path to `php.exe` on your machine. **Important:** take note that when entering filepaths in `settings.json`, you must use the Linux style forward slash (/) rather than the Windows style back slash (\).

```
settings.json
C: > Users > troys > AppData > Roaming > Code > User > settings.json > ...
    "editor.formatOnPaste": true,
    "workbench.tree.indent": 22,
    "workbench.settings.applyToAllProfiles": [
        "workbench.tree.indent"
    ],
    "editor.detectIndentation": false,
    "liveServer.settings.donotShowInfoMsg": true,
    "livePreview.autoRefreshPreview": "On Changes to Saved Files",
    "livePreview.customExternalBrowser": "Chrome",
    "php.format.rules.alignConstants": true,
    "cloudcode.project": "shopify-apps-389118",
    "json.format.keepLines": true,
    "security.workspace.trust.untrustedFiles": "open",
    "diffEditor.ignoreTrimWhitespace": false,
    "html.format.contentUnformatted": "",
    "html.format.unformatted": "script, style",
    "php.validate.executablePath": "D:/Dev/php/php.exe",
    "php.debug.executablePath": "D:/Dev/php/php.exe"
}
```

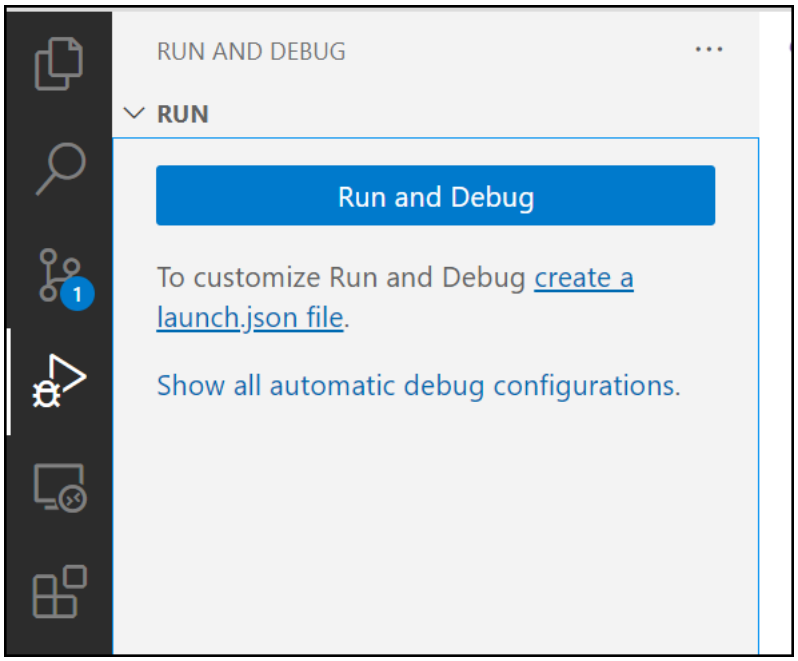
3.6 Lastly, we'll create a simple PHP application to verify that everything is working. Create a new folder (e.g., Lab05-Ex1), open the folder in Visual Studio Code, then add a new file called `index.php`.

3.7 Edit your `index.php` and enter the following code:

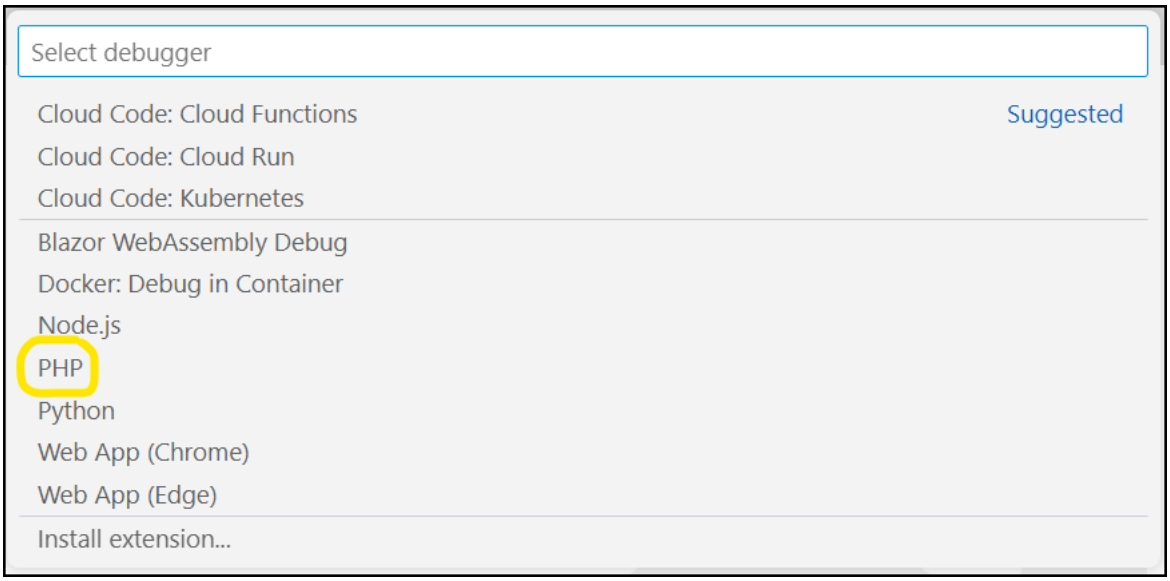
```
<!DOCTYPE html>
<html lang="en">
    <head>
        <meta charset="UTF-8">
        <title>PHP Info</title>
    </head>
    <body>
        <h1>Normal HTML Content...</h1>
        <?php
            echo("<h3>Content inside PHP code...</h3>");
            echo("<hr />");
            phpinfo();
        ?>
    </body>
</html>
```

3.8 Note that `index.php` works like `index.html` in that it's the default file that the server will load if no explicit file is requested. The structure of a PHP file is very similar to HTML, with the addition of the special `<?php ?>` elements. Anything outside of the PHP tags will be treated as normal HTML, while any code written within the tags is executed by the PHP interpreter. Content within the PHP code element is displayed using the `echo` PHP command, and statements must end in a semi-colon (PHP follows C-style syntax).

3.9 Time to run our PHP application. Go to the **Run and Debug** section in VS Code, then click the link that says "create a launch.json file":

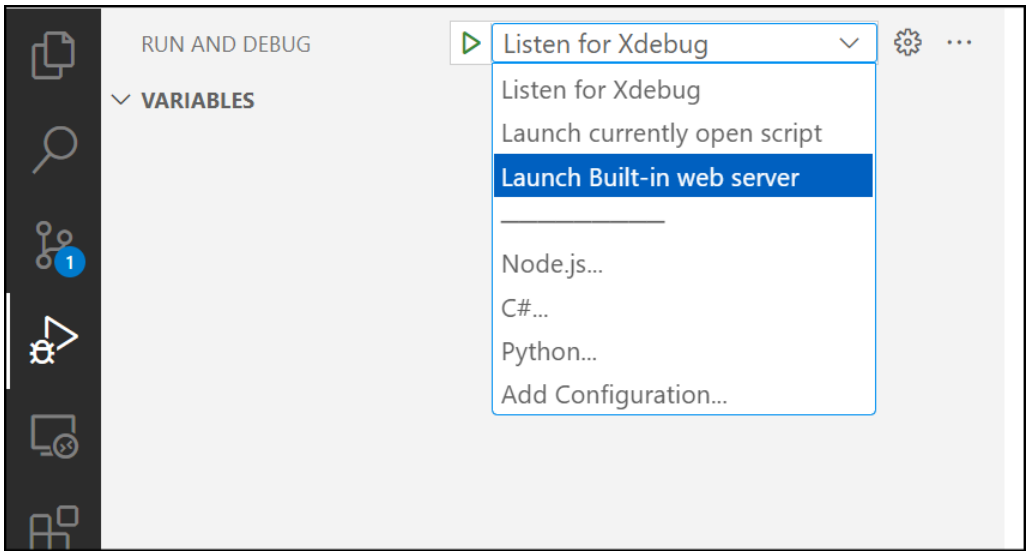


This will auto-generate a launch settings file based on installed extensions. If you have multiple debugging extensions installed, you may be prompted to select one – in this case, choose PHP:

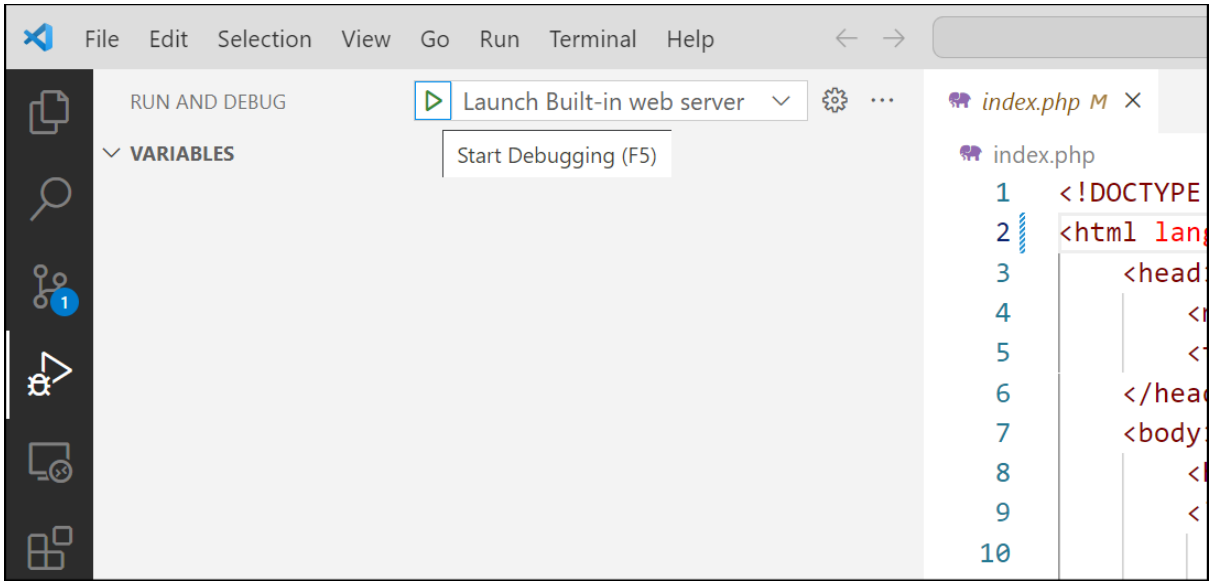


As you can see, the PHP Debug extension that we installed earlier has created a few configurations for us, including "Launch Built-in web server" that we will use most.

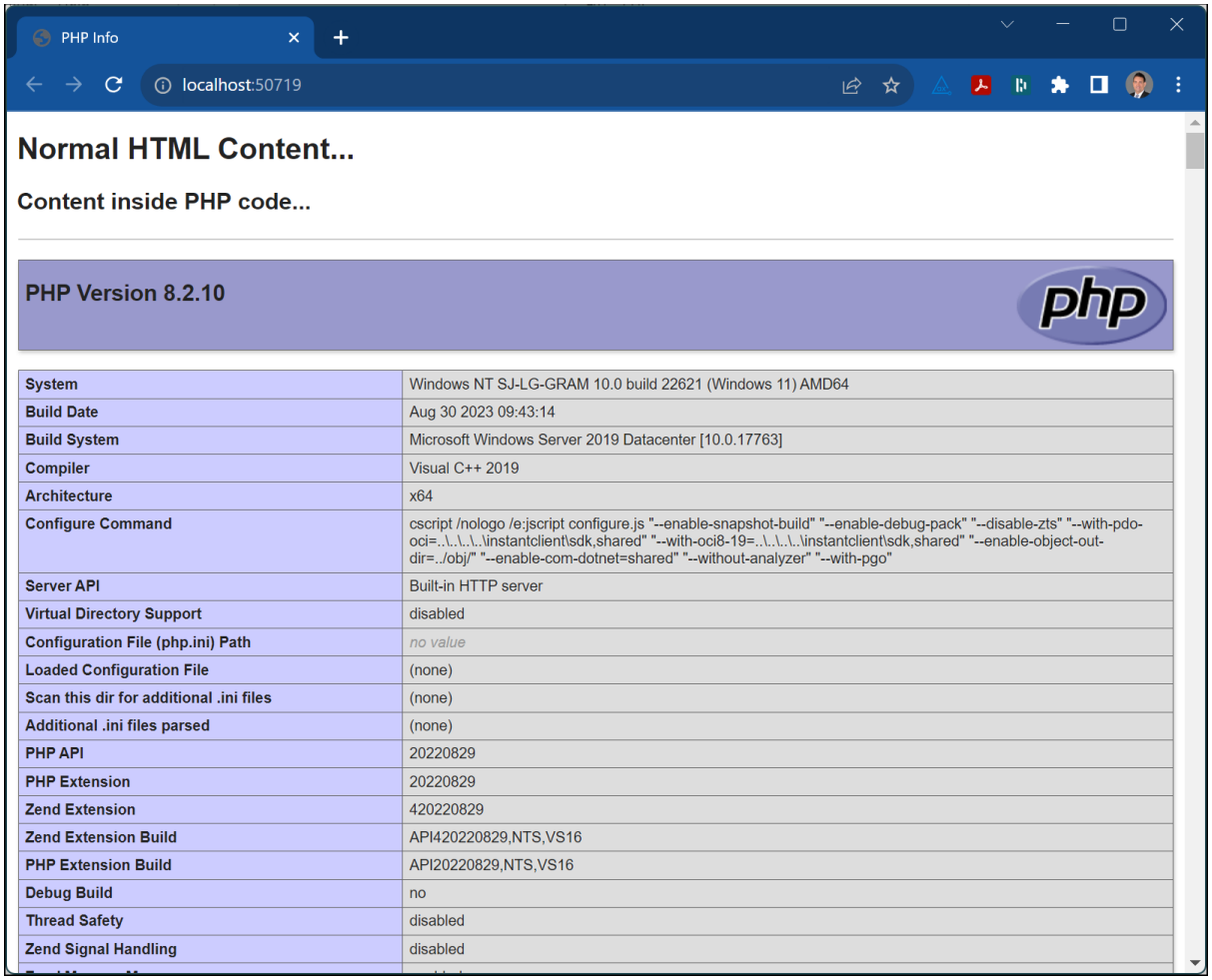
Close the `launch.json` file, then at the top of the **Run and Debug** pane, use the drop-down button to change the configuration to “Launch Built-in web server”:



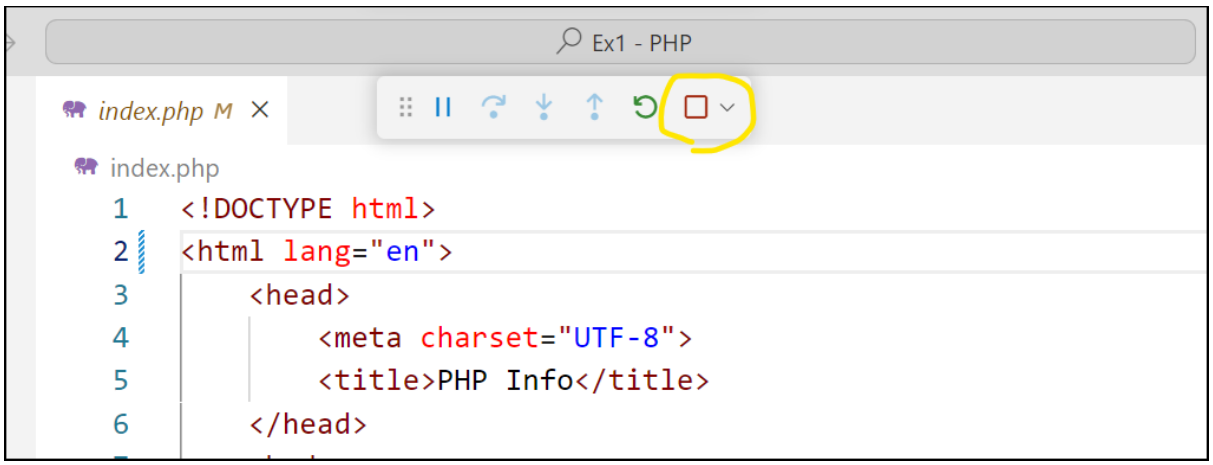
Now click the arrow button (or hit **F5**) to launch the application:



Your default browser should now be showing the PHP Info page:



To stop running/debugging, click the stop button in the VS Code debug toolbar:



And that's it! You're now up and running with PHP development. In the next exercise, you'll learn more about the PHP language and how to debug PHP applications.

Discussion Questions:

a. What are the advantages and disadvantages of server-side development compared with client-side development?

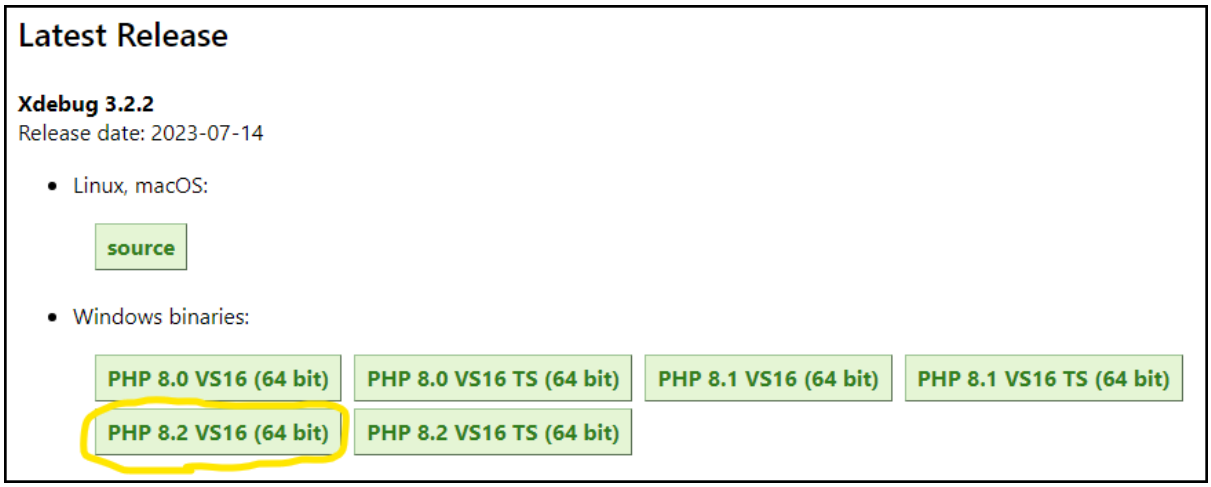
b. What are some other popular server-side development tools and how do they compare with PHP?

4. EXERCISE 2: DEBUGGING PHP WITH XDEBUG

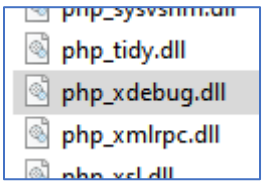
4.1 As we learned with JavaScript previously, knowing how to use debugging tools to find errors in your PHP code will save you lots of time and headaches, especially if you're just learning the language. In this exercise, you will learn how to use the Xdebug extension for PHP.

4.2 Follow these steps to install and configure Xdebug on Windows:

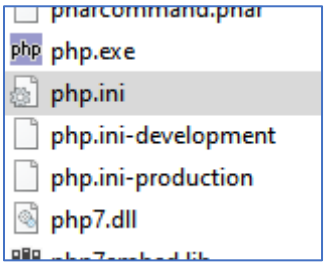
a. Go to the Xdebug download page listed in section 2 and download the version of Xdebug matching your PHP installation. In our example, this is **PHP 8.2 VS16 (64 bit)**. Be sure to select the 64-bit non thread safe version (i.e. *without* the 'TS' in the name). You should be using version 3.2.2 or later of Xdebug.



b. Download the .dll file to the **ext** subfolder under your PHP installation, e.g. `D:\dev\php\ext`. Then rename the file to "**php_xdebug.dll**".



c. Navigate to the main PHP installation folder and look for a file named `php.ini-development`. Make a copy of this file and rename it to `php.ini`.



d. Open `php.ini` in VS Code (or Notepad or your desired text editor) and search for the "Paths and Directories" section. Under this section, find the `extension_dir` setting and remove the comment (`;`) if present, and ensure that `extension_dir` is set to "ext" (for Windows OS):

```
763
764   ; Directory in which the loadable extensions (modules) reside.
765   ; https://php.net/extension-dir
766   ;extension_dir = "./"
767   ; On windows:
768   extension_dir = "ext"
769
```

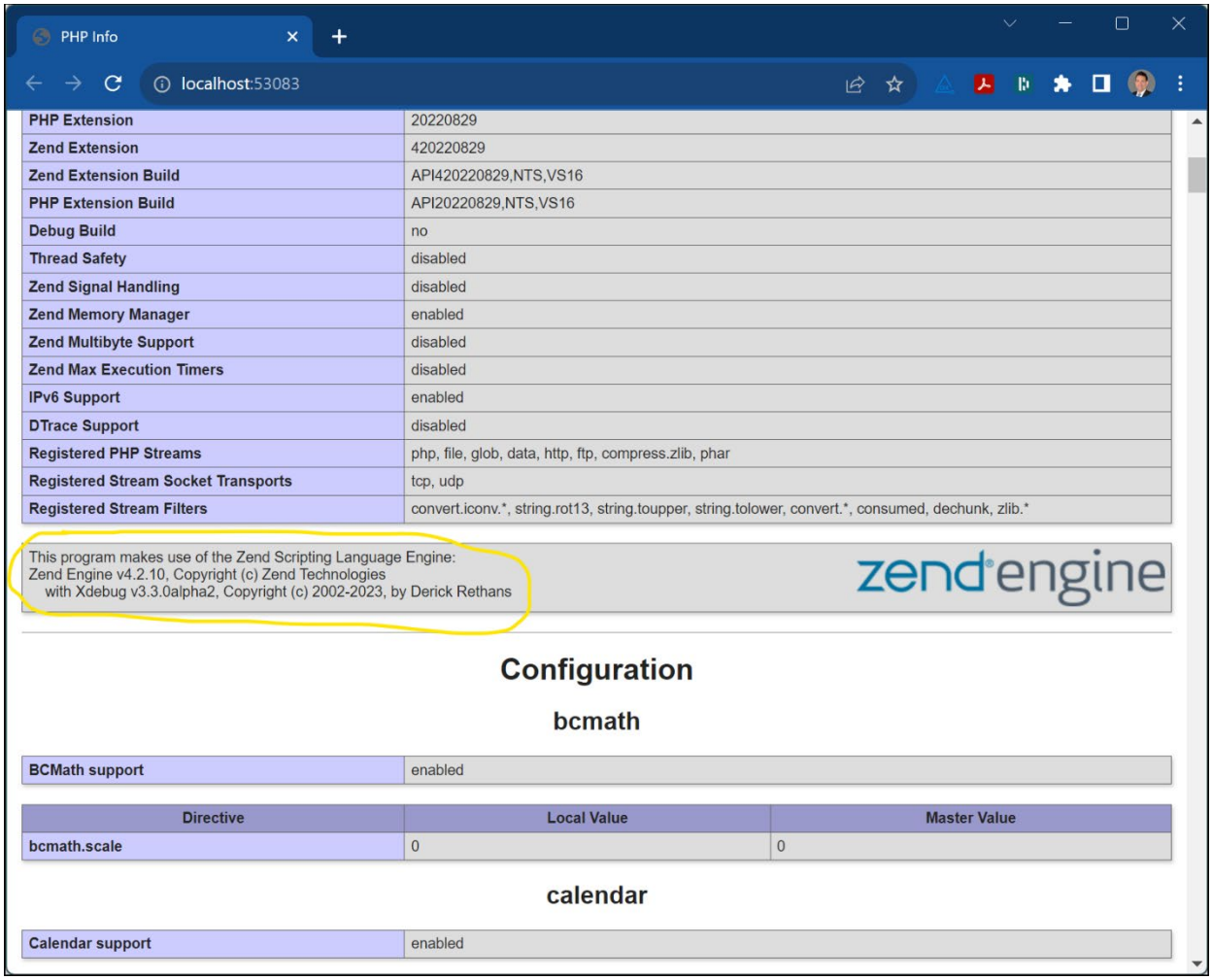
e. Add the following section at the end of the `php.ini` file:

```
[Xdebug]
zend_extension=xdebug
xdebug.mode=debug
xdebug.start_with_request=yes
```

f. Be sure to save and close the file.

4.3 For Mac OS and others, refer to: http://xdebug.org/docs/install, then follow the above steps to edit `php.ini`.

4.4 Now run the PHP application again, following the procedure in Exercise 1. Verify that Xdebug was installed properly by scrolling down and looking to the left of the Zend Engine logo - you should see the Xdebug version information:



4.5 Create a new folder (e.g. Lab05-Ex2) and add an `index.php` file with the following code:

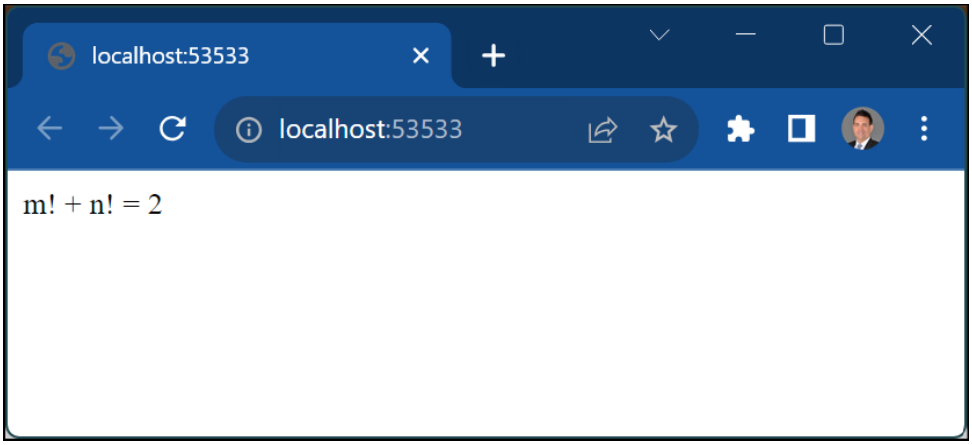
```
<!DOCTYPE html>
<html lang="en">
    <head>
        <meta charset="UTF-8">
        <title></title>
    </head>
    <body>
        <?php
            $m = 5;
            $n = 10;
            $sum_of_factorials = calculate_sum_of_factorials($m, $n);
            echo "m! + n! = " . $sum_of_factorials;

            function calculate_sum_of_factorials($argument1, $argument2)
            {
                $factorial1 = calculate_factorial($argument1);
                $factorial2 = calculate_factorial($argument2);
                $result = calculate_sum($factorial1, $factorial2);
                return $result;
            }

            function calculate_factorial($argument)
            {
                $factorial_result = 1;
                for ($i = 1; $i < $argument; $i++)
                {
                    $factorial_result = $factorial_result;
                }
                return $factorial_result;
            }

            function calculate_sum($argument1, $argument2)
            {
                return $argument1 + $argument2;
            }
        ?>
    </body>
</html>
```

4.6 Repeat the steps in Exercise 1 (step 3.9) to generate the `launch.json` file. Run the application and view the web page in the browser - you should see the following rather suspicious output:



Clearly 5! + 10! does not equal 2, so we have errors. Let's use the debugger to find and fix them.

4.7 Start by setting one or more breakpoints in the PHP code. To set a breakpoint, click to the left of the line number corresponding to the PHP code that you want execution to stop and for Xdebug to take over. The break point is shown by a solid circle in VS Code, as shown below (to remove a breakpoint, click it again):

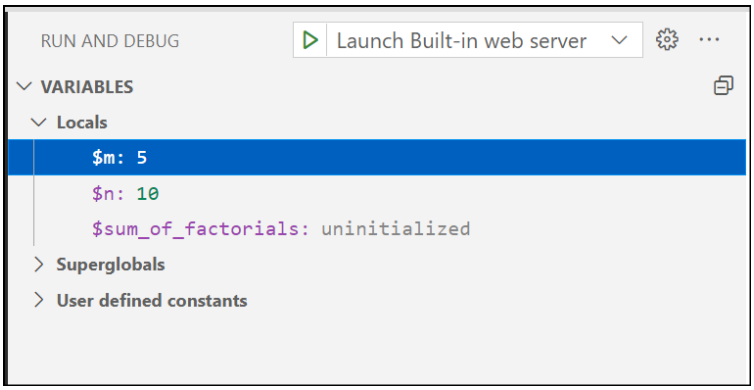
```
index.php M
index.php
1  <!DOCTYPE html>
2  <html lang="en">
3      <head>
4          <meta charset="UTF-8">
5          <title></title>
6      </head>
7      <body>
8          <?php
9              $m = 5;
10             $n = 10;
11             $sum_of_factorials = calculate_sum_of_factorials($m, $n);
12             echo "m! + n! = " . $sum_of_factorials;
13
14             function calculate_sum_of_factorials($argument1, $argument2)
15             {
16                 $factorial1 = calculate_factorial($argument1);
17                 $factorial2 = calculate_factorial($argument2);
18                 $result = calculate_sum($factorial1, $factorial2);
19                 return $result;
```

4.8 While in debugging mode, VS Code will present a toolbar for you to control the debugging processes:

The options include:

- Continue (F5)
- Step Over (F10)
- Step Into (F11)
- Step Out (Shift + F11)
- Restart (Ctrl + Shift + F5)
- Stop (Shift + F5)

Additionally, in the **Run and Debug** panel, you can inspect variable values as you step through the code:



4.9 Click the **Restart** button to reload the web page, and this time execution should stop where you set the breakpoint earlier. Note that nothing will be showing in the browser yet, since the PHP application was paused before generating any content.

4.10 Use the **Step Into** operation to execute the code line by line, observing how the variables change. Notice that when you get to the `calculate_sum_of_factorials()` function, the arrow icon (which indicates the next line to be executed) jumps down to that function. If you had used the **Step Over** operation, the function would execute and return instead.

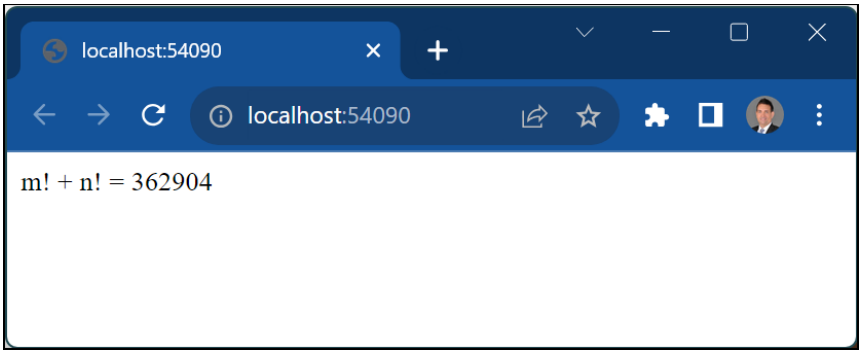
4.11 Continue using **Step Into** until you reach the `calculate_factorial()` function (line #24):

```
7   <body>
8       <?php
9           $m = 5;
10          $n = 10;
11          $sum_of_factorials = calculate_sum_of_factorials($m, $n);
12          echo "m! + n! = " . $sum_of_factorials;
13
14          function calculate_sum_of_factorials($argument1, $argument2)
15          {
16              $factorial1 = calculate_factorial($argument1);
17              $factorial2 = calculate_factorial($argument2);
18              $result = calculate_sum($factorial1, $factorial2);
19              return $result;
20          }
21
22          function calculate_factorial($argument)
23          {
24              $factorial_result = 1;
25              for ($i = 1; $i < $argument; $i++)
26              {
27                  $factorial_result = $factorial_result;
28              }
29              return $factorial_result;
30          }
31
32          function calculate_sum($argument1, $argument2)
33          {
34              return $argument1 + $argument2;
35          }
36      ?>
37  </body>
```

4.12 Step through the execution of the `for` loop and observe how the values of the variables change through each iteration. You should be able to immediately identify one of the problems: the variable `$factorial_result` is always 1, as it's being assigned to itself!

4.13 Stop the debugger by clicking the **Stop** icon, then fix that line of code as follows: `$factorial_result = $factorial_result * $i;`

4.14 Save the file and run the application again to observe the results after fixing this bug:

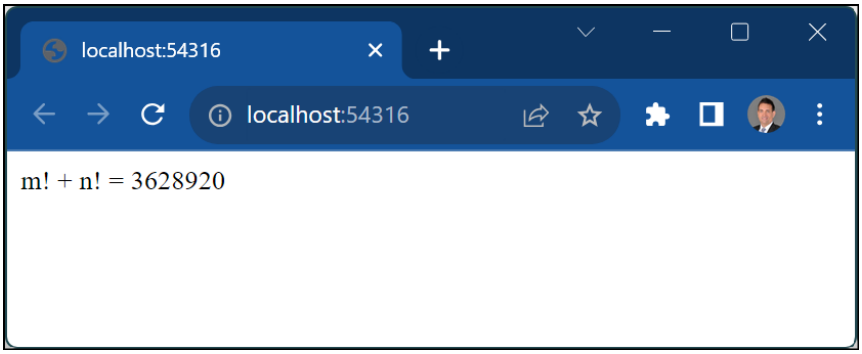


Looks more reasonable this time. But let's check the result using our calculator:

5! = 120
10! = 3,628,800
5! + 10! = **3,628,920**

Something still isn't right. Set a new breakpoint at the start of the `for` loop in the `calculate_factorial()` function (you can remove the breakpoint we set earlier). Run the app in the debugger and step through the loop once again, observing the variable values as they change. Can you find the second bug? **Hint:** this is what's known as a "off-by-one error" and is one of the most common programming mistakes.

After fixing this bug, your output should match with the calculator result:



4.15 You're now ready to go on to more advanced PHP development. Always remember that the debugger is one of the most powerful tools in a developer's toolbox – use it often, it will save you many headaches!

4.16 For more tips on debugging with Visual Studio Code, refer to: https://code.visualstudio.com/docs/editor/debugging

5. EXERCISE 3: CONVERTING AN HTML PROJECT TO PHP

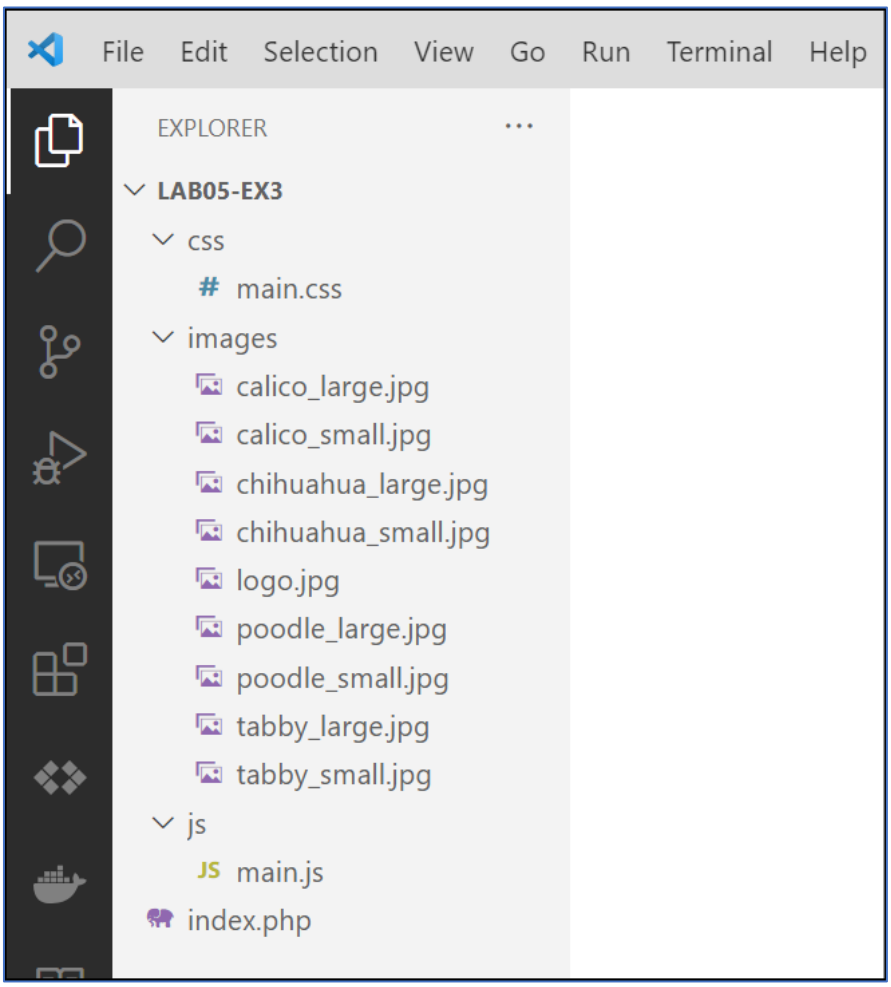
5.1 In this exercise we will "PHP'ify" the World of Pets project that we've been working on in prior labs - that is, we'll convert it from a HTML client-side application to a PHP powered server-side application.

5.2 Create a new folder (e.g. Lab05-Ex3), open it in VS Code, then follow the steps below to migrate your World of Pets website into your new PHP application:

a. Copy all source files from your Lab03 project into the new folder, maintaining the same file structure.

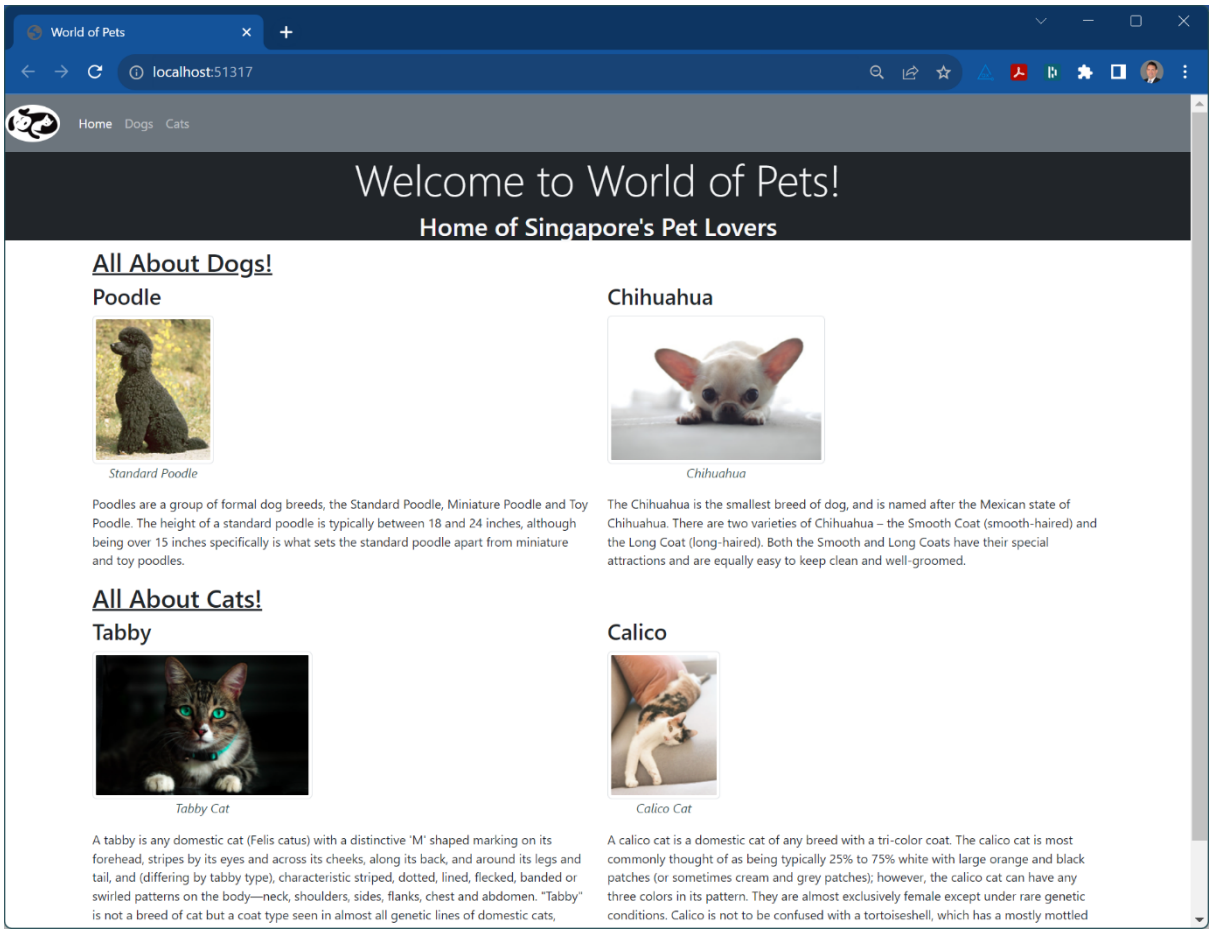
b. Rename `index.html` to `index.php`.

c. Your workspace in VS Code should now look like this:



d. Edit `index.php` and replace all occurrences of `index.html` with `index.php` so that your links all work.

5.3 Repeat the steps in Exercise 1 (step 3.9) to generate the `launch.json` file, then run the application and view the web page in the browser. Be sure the web page looks the same as before, and that all links and pop-ups are working.



6. EXERCISE 4: REUSING CONTENT WITH PHP INCLUDES

6.1 Now that our World of Pets website is PHP'ified, we can take advantage of PHP's page organization features to make it easier to extend and reuse common parts. In the following steps, we'll extract the common elements (the parts that we want to appear on every page in our website) and put them into separate PHP files so that they can easily be reused when we add more pages later. This also makes the site easier to maintain, since you can change common features, such as the navigation menu, in one place.

a. Create a sub-folder in your workspace named `inc`. This is where we will store all of our PHP include files.

b. Add a new PHP file to the `inc` folder and name it `nav.inc.php`. Open `nav.inc.php` in the editor and move (cut & paste) all of the navigation related content from `index.php` into

`nav.inc.php`. If you used the `<nav>` semantic tag previously, then it will make it easy to simply select the entire `<nav>` element and move it to `nav.inc.php`.

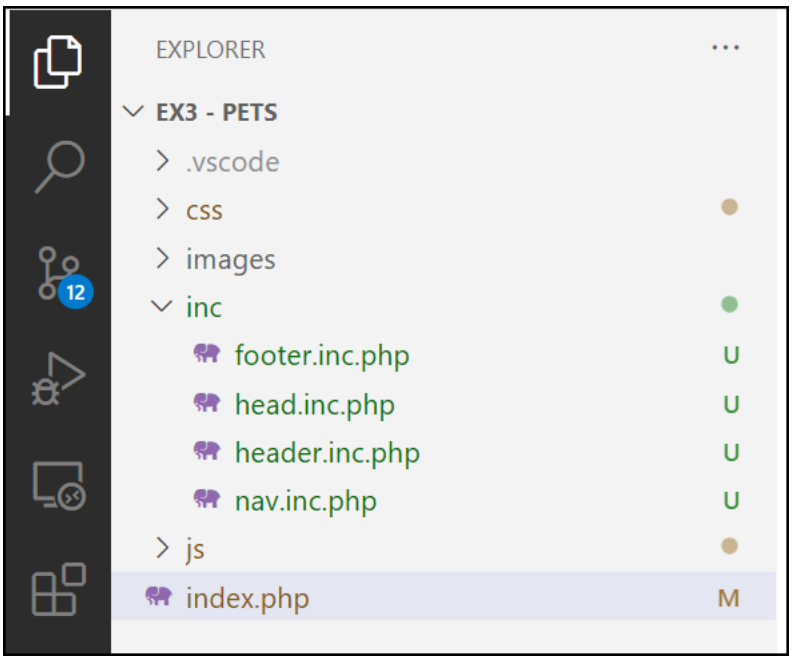
c. Similarly, add another PHP file to the `inc` folder and name it `footer.inc.php`. Then move (cut & paste) all of the footer related content from `index.php` into `footer.inc.php`. Once again, if you used the `<footer>` semantic tag previously, then it should be easy to select the entire `<footer>` element and move it to `footer.inc.php`.

d. If you run your website now, you'll see that the menu and footer are gone, so let's use the PHP `include` function to add them back. Add the following PHP code to `index.php` at the location where you want your navigation menu to appear:

```
<?php
    include "inc/nav.inc.php";
?>
```

e. Do the same for the footer, then run your web application to make sure the menu and footer are appearing as before.

f. Feel free to move other commonly used code into include files. In the following example, portions of the `<head>` element that are common to all pages (such as the CSS and JavaScript elements), and the `<header>` element (containing the Jumbotron) have also been moved to separate PHP include files:



This keeps our `index.php` file clean:

```php
index.php
<!DOCTYPE html>
<html lang="en">
    <head>
        <title>World of Pets</title>
        <?php
            include "inc/head.inc.php";
        ?>
    </head>
    <body>
        <?php
            include "inc/nav.inc.php";
        ?>

        <?php
            include "inc/header.inc.php";
        ?>

```

6.2 In our next Lab, we'll continue to enhance our website by adding forms and server-side processing. Having our code organized this way using PHP includes will make it much easier to maintain a consistent look & feel when adding new files.

7. <u>SUBMISSION OF LAB ASSIGNMENT</u>

7.1 In order to receive credit for this Lab assignment, you must submit your completed work to xSiTe LMS before the end of the Lab session. **Note: you will need to submit all three exercises for today's assignment.** You may either submit three different .zip files, or add all three projects to a single .zip file, whichever you prefer. To submit your work:

a. Save all files and close Visual Studio Code.

b. In File Explorer, navigate to the location where you saved your project and right-click on the folder name, then select 'Send to -> Compressed (zipped) folder' and ZIP up your entire project. **Note: only .zip format is acceptable, do not use .rar, .7z, or any other format.**

c. In the INF1005 module on xSiTe, go to **Assessments->DropBox** and locate the Dropbox folder corresponding to this Lab. Click the link to open the Dropbox then hit the **Add a File** button to submit your .zip file. You may also add comments if desired. Be sure to hit **Submit** to complete your submission.

d. Remember to save a copy of your work as we will be building upon this website in subsequent Lab assignments.

8. ADDITIONAL PRACTICE

8.1 Once you've completed this Lab assignment, you are encouraged to check out the following references:

a. PHP in Visual Studio Code: https://code.visualstudio.com/docs/languages/php

b. PHP Manual: https://www.php.net/manual/en/index.php

c. PHP tutorials: https://www.w3schools.com/php/default.asp

d. Debugging with Visual Studio Code: https://code.visualstudio.com/docs/editor/debugging